Generating PDFs in Java: four libraries compared

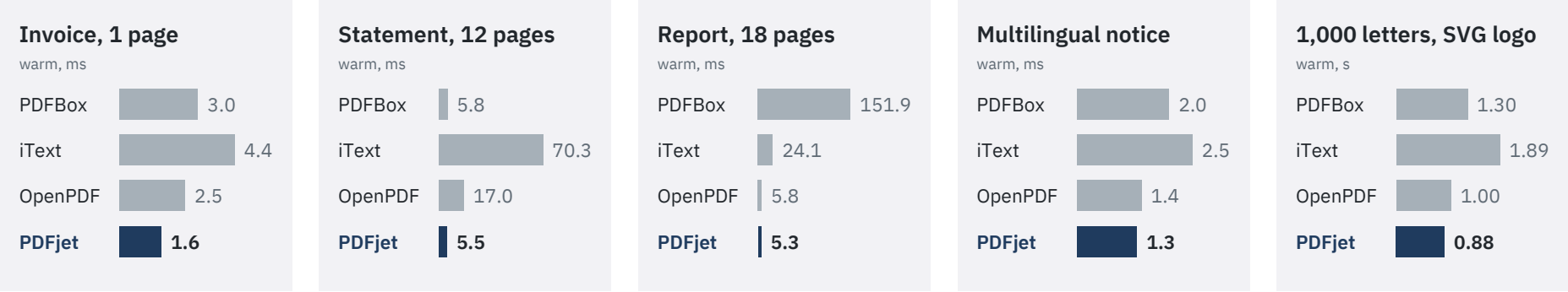
Five business documents and a batch of letters, made by each library the way its own documentation shows, every PDF checked, in **IBM Plex Sans**, and all of them again in **Noto Sans**. The other measurements, the features of each library, the method and the tables of every figure are on the pages after this one.

I wrote PDFjet, one of the four libraries compared here, and have been updating it since 2001. For the last two months I have worked on it using Claude. I commissioned this benchmark; Claude chose the documents, their content and the way each library is used, wrote the programs, ran them and wrote the tables. Both are reasons to be suspicious of the results, so everything needed to check them is published in github.com/edragoev1/pdf-benchmarks: the specification of the documents, every program, the scripts that run and check them, and the raw results.

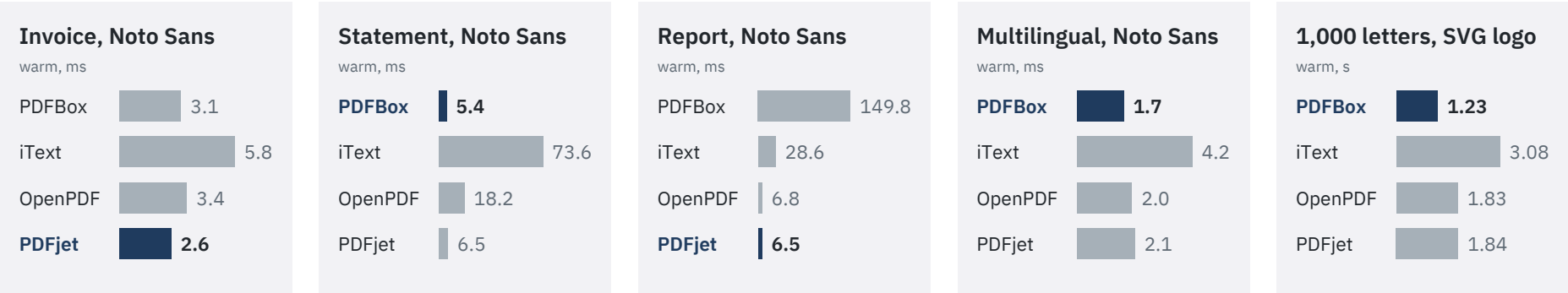
If we have configured a library badly, or used it in a way its users would not, please tell us or open an issue, and we will fix the configuration and run the benchmark again.

Lower is better, and the best is in navy. Each chart starts at zero and is scaled to its own largest value.

Time per document. These tests use IBM Plex Sans.



Time per document, Noto Sans. These tests use Noto Sans, which has many more glyphs than IBM Plex Sans.



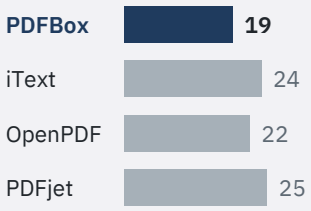
The size of the PDFs and memory

Lower is better, and the best is in navy. Each chart starts at zero and is scaled to its own largest value.

The size of the PDFs. These tests use IBM Plex Sans.

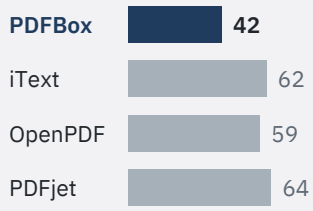
Size, the invoice

KB



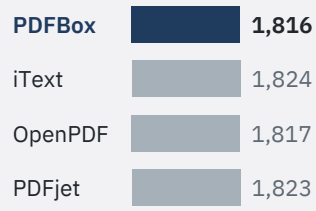
Size, the statement

KB



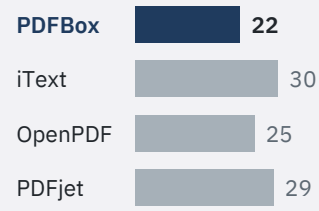
Size, the report

KB



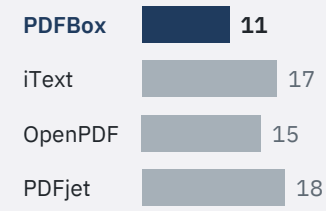
Size, the notice

KB



Size, one letter

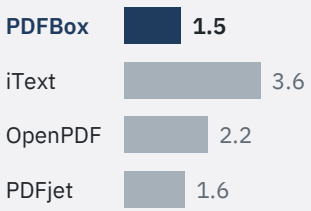
KB



Memory allocated per document. These tests use IBM Plex Sans.

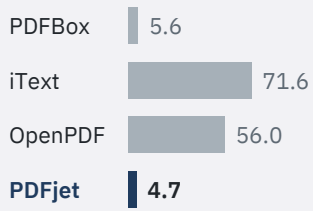
Memory, the invoice

MB allocated



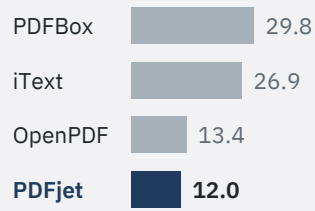
Memory, the statement

MB allocated



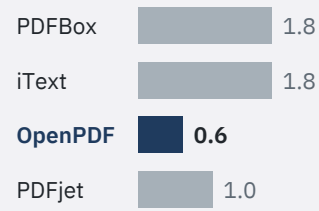
Memory, the report

MB allocated



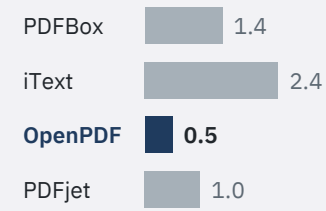
Memory, the notice

MB allocated



Memory, one letter

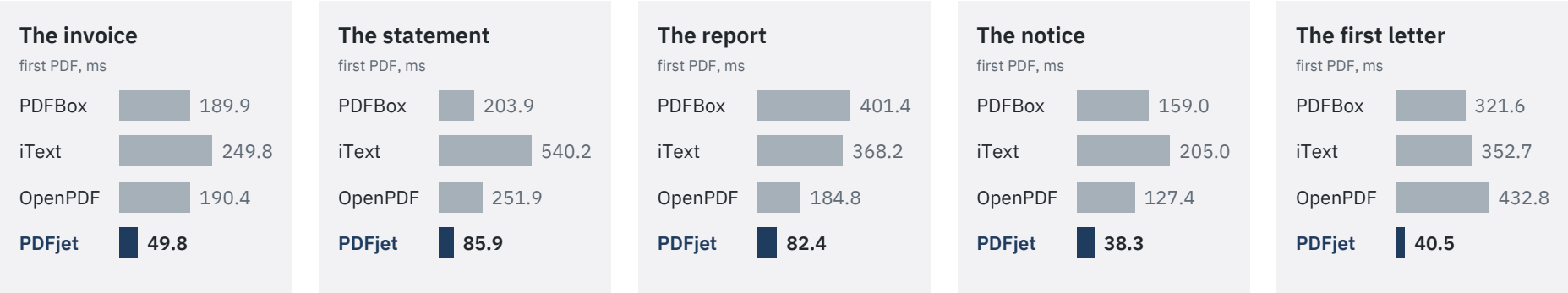
MB allocated



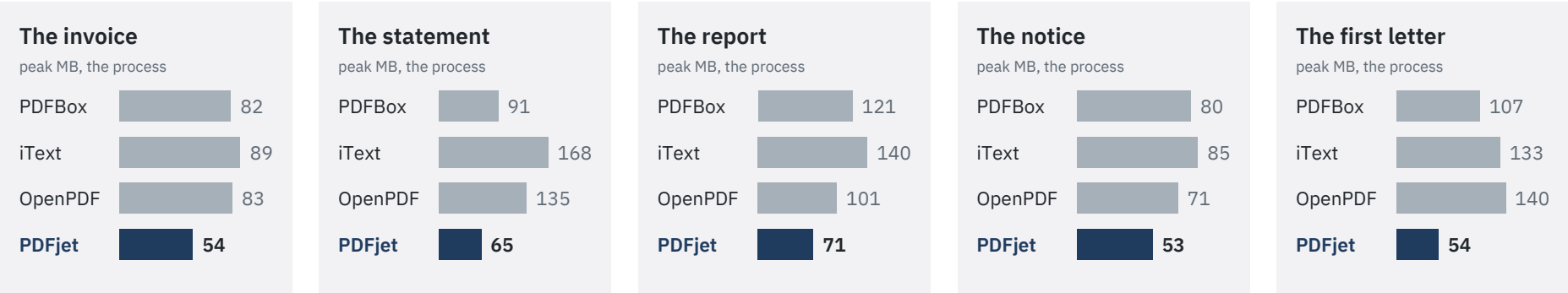
The first PDF of a fresh process

Lower is better, and the best is in navy. Each chart starts at zero and is scaled to its own largest value.

The first PDF of a fresh process. These tests use IBM Plex Sans.



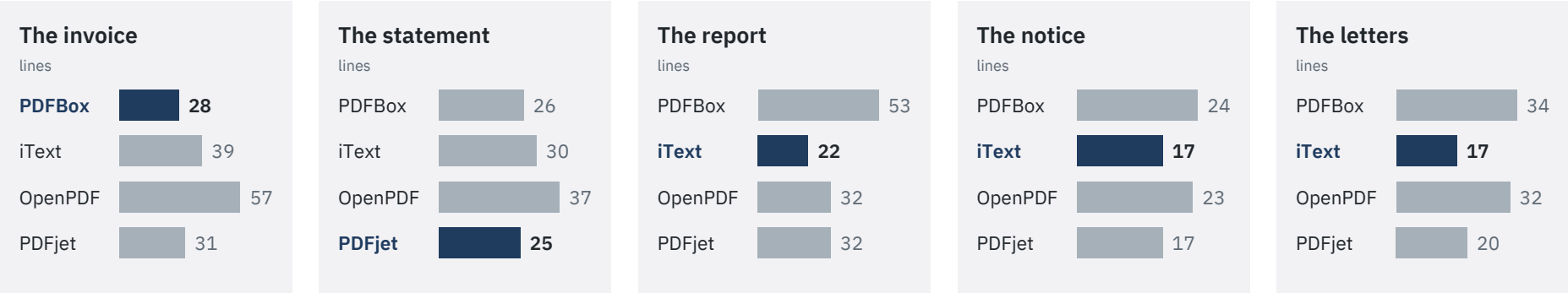
The peak memory of a fresh process, making its first PDF. These tests use IBM Plex Sans.



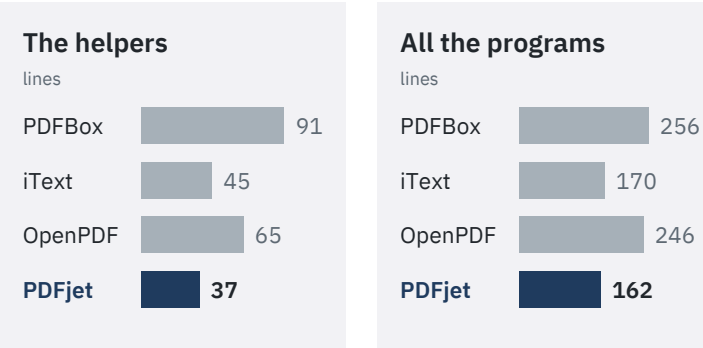
The lines of code of the programs

Lower is better, and the best is in navy. Each chart starts at zero and is scaled to its own largest value.

Lines of code of each program. Without blank lines and comments.



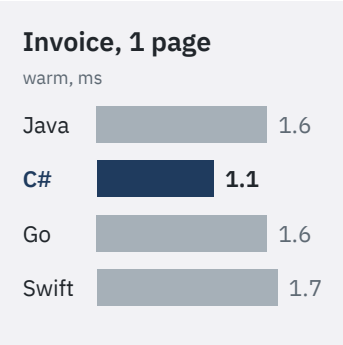
The helpers the programs share, and the total. The helpers are counted in the total.

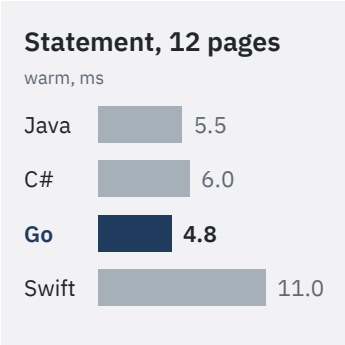


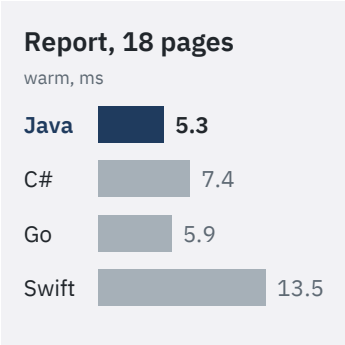
PDFjet in Java, C#, Go and Swift

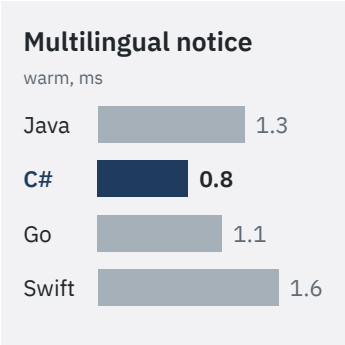
Lower is better, and the best is in navy. Each chart starts at zero and is scaled to its own largest value.

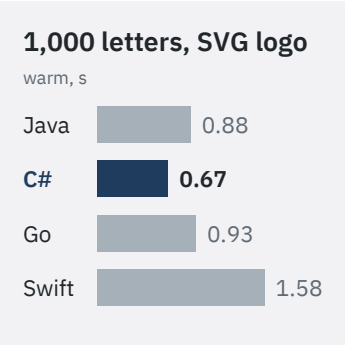
Time per document, the four ports. These tests use IBM Plex Sans.



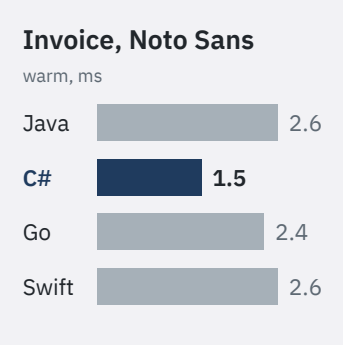


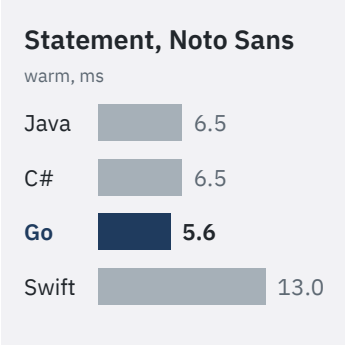


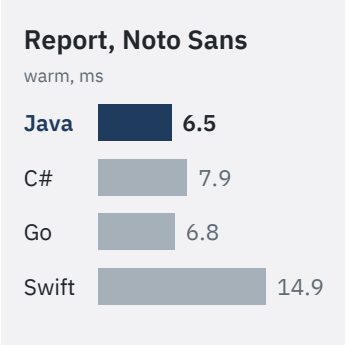


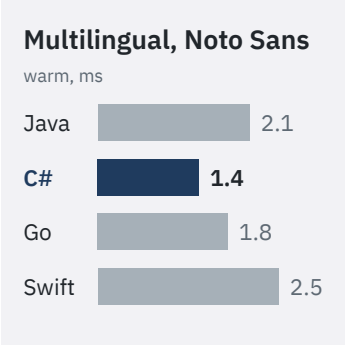


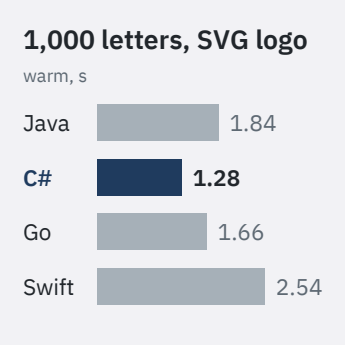
Time per document, Noto Sans, the four ports. These tests use Noto Sans, which has many more glyphs than IBM Plex Sans.







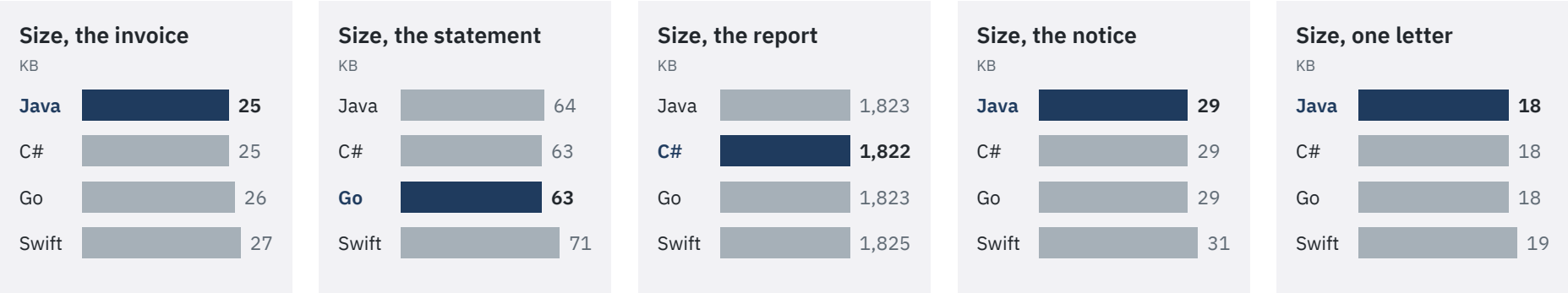




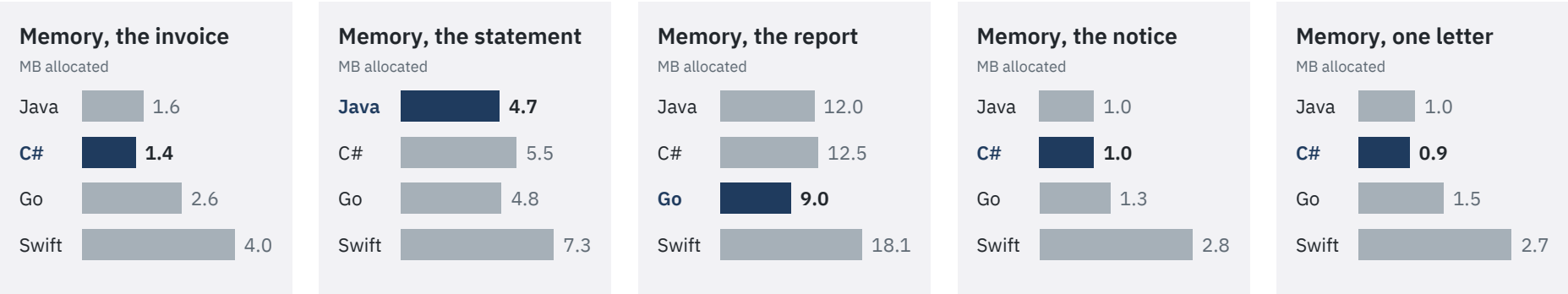
PDFjet in Java, C#, Go and Swift: the size of the PDFs and memory

Lower is better, and the best is in navy. Each chart starts at zero and is scaled to its own largest value.

The size of the PDFs, the four ports. These tests use IBM Plex Sans.



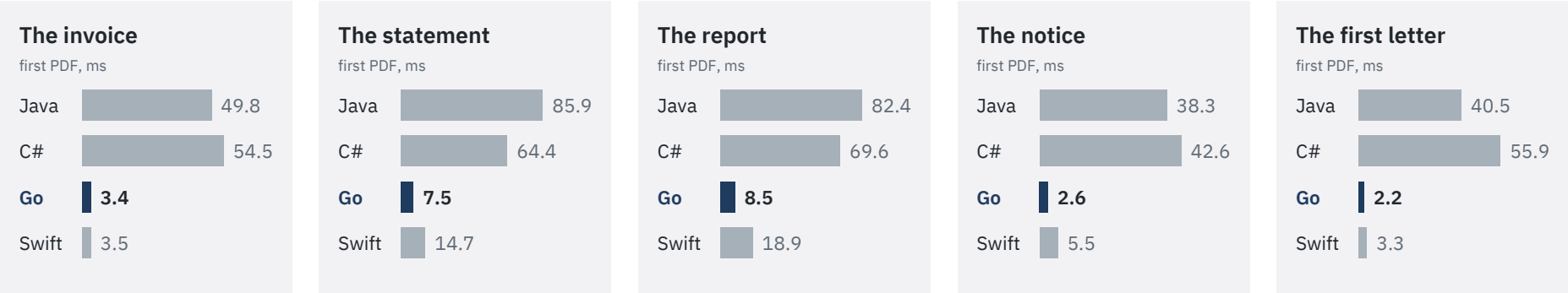
Memory allocated per document, the four ports. These tests use IBM Plex Sans.



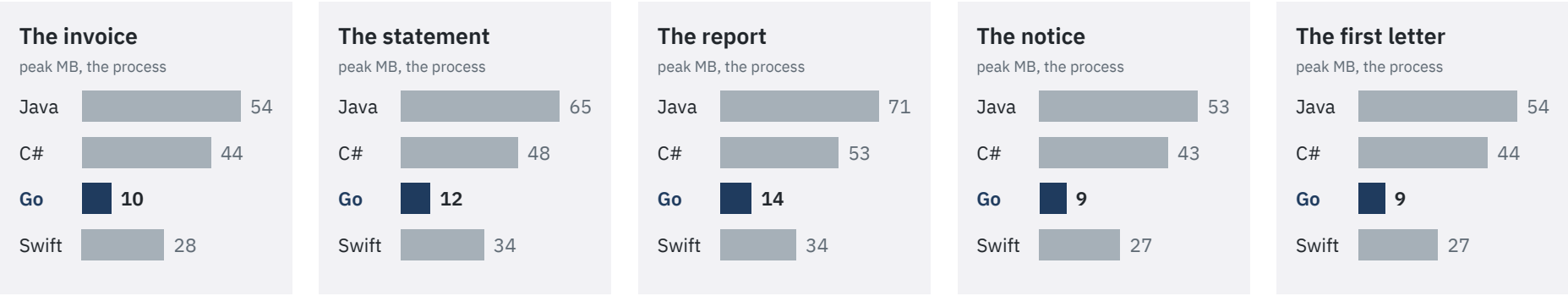
PDFjet in Java, C#, Go and Swift: the first PDF

Lower is better, and the best is in navy. Each chart starts at zero and is scaled to its own largest value.

The first PDF of a fresh process, the four ports. These tests use IBM Plex Sans.



The peak memory of a fresh process, making its first PDF. These tests use IBM Plex Sans.



The libraries

Library	Version	License	Java
PDFjet (core)	9.1.7	MIT	8 and later
Apache PDFBox	3.0.8	Apache 2.0	8 and later
iText Core	9.8.0	AGPL-3.0 or commercial	8 and later
OpenPDF	3.0.5	LGPL-2.1 or MPL-2.0	21 and later

The newest releases on Maven Central on 5 November 2026. For the letters with an SVG logo, the modules the others need to draw SVG, each at its newest release: iText's svg module 9.8.0, Apache Batik 1.20, and for PDFBox the pdfbox-graphics2d bridge 3.0.5.

Each library is used the way its own documentation shows: iText's and OpenPDF's `Document` for flowing text, PDFBox's content streams with lines wrapped by hand, as it has no layout of its own, and PDFjet's TextFrame and Table, and for the report TextFlow. Every program reads the font files from disk once, into memory, and each document makes its fonts from those bytes, as a server would: iText and OpenPDF with their own default caches, PDFBox with its TrueType font parsed once and reused, as its API allows; PDFjet has no font cache and parses the bytes for each document. We ran iText both with its font cache and without it: without it, iText takes 1 to 5 ms more per document. The results here are with font caching.

Features

What each library has, as of the versions above, from its source at the release tag, its Maven POM and its documentation; only the features the documents use were tested here. A module is another jar of the same project; an add-on is a separate product of the same company.

PDFBox has no table of its own, so its programs here draw the tables by hand: a gray box behind the header, each cell's text measured to align the numbers right, a rule under each row, the header again on each new page, and the page numbers in a second pass once the pages are counted.

The other jars each of the others needs at run time, from its POM: PDFBox pdfbox-io, fontbox and commons-logging; iText io, commons, jackson-databind, slf4j-api and bouncy-castle-connector, and its layout jar for tables and flowing text; OpenPDF none: Maven adds brotli4j, its service jar and a native jar for the platform, needed only for Brotli compression (OpenPDF ran here without them). pdfHTML, pdfSweep and pdfOCR are iText add-ons under the same terms as iText Core, AGPL or commercial; pdfCalligraph, pdfRender and pdfXFA are commercial only. PDFjet Pro is PDFjet's add-on, commercial.

Far more people have worked on PDFBox, iText and OpenPDF than on PDFjet, and far more people use them, with more answers on the web for every question; iText also has a company behind it with paid support.

Feature	PDFBox	iText	OpenPDF	PDFjet
Tables	no	layout module	✓	✓
Text wrapped into paragraphs	no, by hand	layout module	✓	✓
Text flowing onto new pages	no	layout module	✓	✓
Tagged PDF, PDF/UA-1	by hand: your code builds the structure tree, numbers the marked content and builds the parent tree	✓ tags; pdfua module for PDF/UA	by hand: your code marks the content of each element; OpenPDF builds the parent tree	✓
PDF/UA-2	no	pdfua module	no	no
PDF/A	by hand; xmpbox module for the metadata	pdfa module, 1a to 4f	1a to 3u	1a to 3b
SVG	through Apache Batik and pdfbox-graphics2d	svg module	through Apache Batik	✓
Barcodes, 1D	no	barcodes module	✓	✓
QR codes	no	barcodes module	no	✓
Data Matrix	no	barcodes module	✓	✓
PDF417	no	barcodes module	✓	✓
Charts	no	no	no	bar, line, scatter, donut
Markdown to PDF	no	no	no	✓
HTML to PDF	no	pdfHTML add-on	openpdf-html module	no
Images	JPEG, PNG, GIF, BMP, TIFF	those, WMF, JPEG 2000, JBIG2; WebP with the webp-image-support module	those, WMF, JPEG 2000	JPEG, PNG, BMP

Feature	PDFBox	iText	OpenPDF	PDFjet
Kerning	no	pdfCalligraph, commercial	the old kern table only, which neither font here has; GPOS through Java AWT or Apache FOP	✓
Ligatures (GSUB)	✓ by default, turned off here	pdfCalligraph, commercial	through Java AWT or Apache FOP	no
Arabic	no	pdfCalligraph, commercial	✓	✓
Devanagari and other Indic scripts	Devanagari, Bengali, Gujarati, without mark positioning	pdfCalligraph, commercial	through Java AWT or Apache FOP	no
Hyphenation	no	hyph module	✓ patterns not included	soft hyphens; five languages in Pro, commercial
Extract text	✓	✓	✓	no
Render a page to an image	✓	pdfRender, commercial	openpdf-renderer module	no
Change existing PDFs	✓	✓	✓	add content, merge, split
Fill or create form fields	✓	forms module	✓	no
Digital signatures	✓ with Bouncy Castle	sign module, with Bouncy Castle	✓ with Bouncy Castle, PKCS #7, not PAdES	Pro, commercial
Long-term signatures, PAdES B-LT and B-LTA	by copying its example code (AddValidationInformation), not in the library	sign module	no	Pro, commercial

Feature	PDFBox	iText	OpenPDF	PDFjet
Password encryption, AES-256	✓	✓	✓	✓
Certificate encryption	✓ with Bouncy Castle	✓ with Bouncy Castle	✓ with Bouncy Castle	no
PDF 2.0	a 2.0 header, setVersion	✓	a 2.0 header by default	no, PDF 1.7
Object streams, for smaller files	✓ by default	✓ an option	✓ an option	no
Redaction	no	pdfSweep add-on	no	no
OCR	no	pdfOCR add-on	no	no
XFA forms	reads the data	fills the data, forms module; flattens, pdfXFA, commercial	reads, fills the data	no
Electronic invoices: ZUGFeRD, Factur-X, XRechnung	by hand: PDF/A-3 with the XML attached	by hand, with the pdfa module	by hand	Pro, commercial
Check existing files for PDF/A	preflight module, PDF/A-1b	no	no	no
No other jars needed	no	no	✓; Maven adds brotli4j, needed only for Brotli compression	✓
Java	8 and later	8 and later	21 and later	8 and later
Also for	—	.NET	—	.NET, Go, Swift
License	Apache 2.0	AGPL-3.0 or commercial	LGPL-2.1 or MPL-2.0	MIT

What was measured

Five tasks, specified exactly in SPEC.md, the same text and the same layout in every library:

- **invoice**: 1 page, the PDFjet logo, two address blocks, a table of 15 rows, totals.
- **statement**: 12 pages, a table of 600 transactions with running footers "Page n of N".
- **report**: 18 pages, 120 paragraphs of flowing text and 4 photos, JPEGs of 1870 × 2420 pixels, 1.8 MB in all.
- **multilingual**: 1 page, a notice in eight languages: English, French, German, Polish, Turkish, Greek, Bulgarian and Ukrainian.
- **letters**: 1,000 one-page letters with the logo as PNG, and again as SVG, timed as the batch (with Noto, the SVG logo only).

All in **IBM Plex Sans**, embedded as subsets, and all again in **Noto Sans**, a much larger font.

Method. Each library in its own fresh JVM, in a random order, three rounds; within a JVM the tasks run in the same fixed order for every library. Each task is warmed up for at least 20 runs and 3 seconds, then timed over 15 runs (the letters: 2 warm-up batches, 5 timed); the figure is the median of the three round medians. An asterisk (*) marks a figure whose rounds differ by more than 5%. The first PDF of a fresh process is the median of 5 processes. OpenJDK 25, .NET 10, Go 1.27, Swift 6.4, on an AMD Ryzen 5 5600G with Linux, after a reboot, in the performance power profile, nothing else running. After the run every PDF is checked, and the log is published: MuPDF opens it with no warning, it has the expected pages, the check strings of SPEC.md are in its text (the last sentence of every language for the notice), both fonts are embedded as subsets, and qpdf --check passes.

Results

Every measurement is drawn as a chart on the first pages, and the tables of every figure, with PDFjet in C#, Go and Swift beside the four libraries, are in the appendix at the end.

To draw the SVG logo the others need another module or library, and each was given the one its users would choose:

Library	to draw SVG it needs
PDFBox	Apache Batik and pdfbox-graphics2d
iText	iText's svg module
OpenPDF	Apache Batik
PDFjet	nothing else

PDFjet in C#, Go and Swift

PDFjet is also written in C#, Go and Swift: the same classes and the same public API in each, and the same documents laid out the same way, which an automated check compares on every change. The last three columns of the tables in the appendix are the same documents made by each of them. For a program that makes PDFs, what counts is what the port of its language makes, and how fast: that is what those columns show.

Each port compresses its pages with the Deflate its language offers, but for Swift: Swift offers none on Linux, so the Swift port has an encoder of its own, which searches less than zlib. Its PDFs, the largest of the four, are larger because of that encoder, not because of Swift.

Reproduce it

Everything is in github.com/edragoev1/pdf-benchmarks: SPEC.md, the text of the notice, the logo and the photos, the program of each library and port, the run and check scripts, and the raw results with their logs. java/run.sh runs the four Java libraries (about 15 minutes) and java/check.sh checks every PDF; run-all.sh runs everything, as for these results; dotnet/, go/ and swift/ run PDFjet's other ports the same way; make-tables.py makes every table of this report from the results.

Judgment calls

- The table rows of every library are 13.2 pt high, PDFjet's default for IBM Plex Sans at 9 pt; iText's and OpenPDF's cell padding was adjusted to match it, as without it their page counts differ.
- OpenPDF uses `setStrictImageSequence(true)`, or it moves text above a photo that goes to the next page.
- Every library is at its default compression: all four compress their streams, and PDFBox alone writes object streams, its default, which make its files smaller; iText and OpenPDF can write them too, as an option.
- PDFBox shapes ligatures (GSUB) by default on every showText, a cost that grows with the font's tables; the others do none, so it is turned off here (setEnableGsub(false)), as a PDFBox user who wants speed does. In the run of 26 October, with it on, PDFBox took 43 ms for the statement in Plex and 524 ms in Noto; without it, 5.8 and 5.4 ms.
- iText and OpenPDF use their large-table modes for the statement, as their documentation shows for long tables. iText's ordinary Table was timed too, and was slower (about 106 ms against 72, in the run of 26 October).
- PDFBox's report time is almost all in reading the four photos: `createFromByteArray`, as its documentation shows, decodes each JPEG (about 38 ms here) to read its size and colors, then embeds its bytes unchanged. Building the image object by hand with `DCTDecode` avoids it; few users do.
- PDFBox draws one rule under each table row and one header rectangle, where the others draw a border under each cell: slightly less work for PDFBox.

- With Noto Sans, PDFjet's report is 21 pages to the others' 20, and its statement 13 to their 12: PDFjet spaces lines and table rows by the font's own height, and Noto's is larger than Plex's, so PDFjet makes a page more of each. With Plex every library makes the same pages (java/check.sh prints the counts side by side).

Appendix: the tables

Time per document, IBM Plex Sans

Time per document, warm, ms (the letters: the batch of 1,000, s). In every table lower is better, faster, less memory, smaller files, fewer lines, and the best of the four Java libraries is in bold. The last three columns are PDFjet in C#, Go and Swift, shown with them but not ranked against them.

Document	PDFBox	iText	OpenPDF	PDFjet Java	PDFjet C#	PDFjet Go	PDFjet Swift
invoice, 1 page	3.0 *	4.4	2.5	**1.6**	1.1 *	1.6	1.7 *
statement, 12 pages	5.8 *	70.3 *	17.0	**5.5**	6.0 *	4.8	11.0
report, 18 pages	151.9	24.1	5.8	**5.3**	7.4	5.9 *	13.5
multilingual, 1 page	2.0	2.5	1.4	**1.3**	0.8 *	1.1 *	1.6
the 1,000 letters, PNG logo, s	2.49	1.97	1.91	**1.35**	1.03	1.37	1.50
the 1,000 letters, SVG logo, s	1.30	1.89	1.00	**0.88**	0.67	0.93	1.58

* The three rounds of this figure differ by more than 5%; the figure is their median.

Time per document, Noto Sans

With Noto Sans, a large font, ms (the letters: the batch of 1,000, s):

Document	PDFBox	iText	OpenPDF	PDFjet Java	PDFjet C#	PDFjet Go	PDFjet Swift
invoice	3.1 *	5.8	3.4	**2.6**	1.5	2.4 *	2.6 *
statement	**5.4 ***	73.6 *	18.2	6.5	6.5	5.6	13.0
report	149.8	28.6	6.8	**6.5**	7.9 *	6.8 *	14.9
multilingual	**1.7**	4.2	2.0	2.1	1.4 *	1.8 *	2.5
the 1,000 letters, SVG logo, s	**1.23**	3.08	1.83	1.84	1.28	1.66	2.54

* The three rounds of this figure differ by more than 5%; the figure is their median.

The size of the PDFs

The size of the PDFs, KB:

Document	PDFBox	iText	OpenPDF	PDFjet Java	PDFjet C#	PDFjet Go	PDFjet Swift
invoice	**19**	24	22	25	25	26	27
statement	**42**	62	59	64	63	63	71
report	**1,816**	1,824	1,817	1,823	1,822	1,823	1,825
multilingual	**22**	30	25	29	29	29	31
one letter	**11**	17	15	18	18	18	19

Memory allocated per document

Memory allocated per document, MB. Each runtime counts it its own way: Java and C# the bytes the timing thread allocates, Go every allocation of the process (Go's TotalAlloc), Swift every malloc of the thread:

Document	PDFBox	iText	OpenPDF	PDFjet Java	PDFjet C#	PDFjet Go	PDFjet Swift
invoice	**1.5**	3.6	2.2	1.6	1.4	2.6	4.0
statement	5.6	71.6	56.0	**4.7**	5.5	4.8	7.3
report	29.8	26.9	13.4	**12.0**	12.5	9.0	18.1
multilingual	1.8	1.8	**0.6**	1.0	1.0	1.3	2.8
one letter	1.4	2.4	**0.5**	1.0	0.9	1.5	2.7

The first PDF of a process

The first PDF of a fresh process, each of the five documents, as a command-line tool, a serverless function or a server just started pays it, ms; then the peak memory of the whole process, mostly the JVM itself, MB:

	PDFBox	iText	OpenPDF	PDFjet Java	PDFjet C#	PDFjet Go	PDFjet Swift
the invoice, ms	189.9	249.8	190.4	**49.8**	54.5	3.4	3.5
the statement, ms	203.9	540.2	251.9	**85.9**	64.4	7.5	14.7
the report, ms	401.4	368.2	184.8	**82.4**	69.6	8.5	18.9
the notice, ms	159.0	205.0	127.4	**38.3**	42.6	2.6	5.5
the first letter, ms	321.6	352.7	432.8	**40.5**	55.9	2.2	3.3
the invoice, the whole process, ms	240	300	230	**80**	80	under 10	10
the invoice, peak memory, MB	82	89	83	**54**	44	10	28
the statement, peak memory, MB	91	168	135	**65**	48	12	34
the report, peak memory, MB	121	140	101	**71**	53	14	34
the notice, peak memory, MB	80	85	71	**53**	43	9	27
the first letter, peak memory, MB	107	133	140	**54**	44	9	27

Lines of code

Lines of code of each program, without blank lines and comments; "helpers" are the methods a library's tasks share (fonts, cells, footers, word wrapping), counted in the total:

	invoice	statement	report	letters	multilingual	helpers	total
PDFBox	**28**	26	53	34	24	91	256
iText	39	30	**22**	**17**	**17**	45	170
OpenPDF	57	37	32	32	23	65	246
PDFjet Java	31	**25**	32	20	**17**	**37**	**162**
PDFjet C#	32	25	32	20	17	35	161
PDFjet Go	36	26	35	26	20	35	178
PDFjet Swift	29	22	31	18	15	45	160